NATIONAL PARK - NETWORKS AND ASSIGNMENT

In this chapter, we will examine three types of problems:

1. In allocation problems, a limited resource "supply" is allocated to various entities "demand". We will see what happens if demand exceeds supply and vice versa.
2. Assignment problems assign exactly one of n task to each of exactly n workers
3. Network problems look at a variety of issues arising in a network of nodes and arcs, from spanning trees to traversal problems.

Learning Goals

- Allocation problems
- Assignment problems
- Shortest path problem
- Flow problems
- Maximum flow
- Maximum flow - minimum cost
- Traveling salesman problem
- Using Excel to solve the above

Case Study Description

A very popular National Park is facing a number of challenges and has hired your company to propose solutions to the following:

1. Due to budget cuts, much of the work in the park such as interpretive services and trail maintenance is done by volunteers. What is the most efficient way to assign them to the various tasks and locations? Also, you need to make sure each location has one full-time ranger on site.

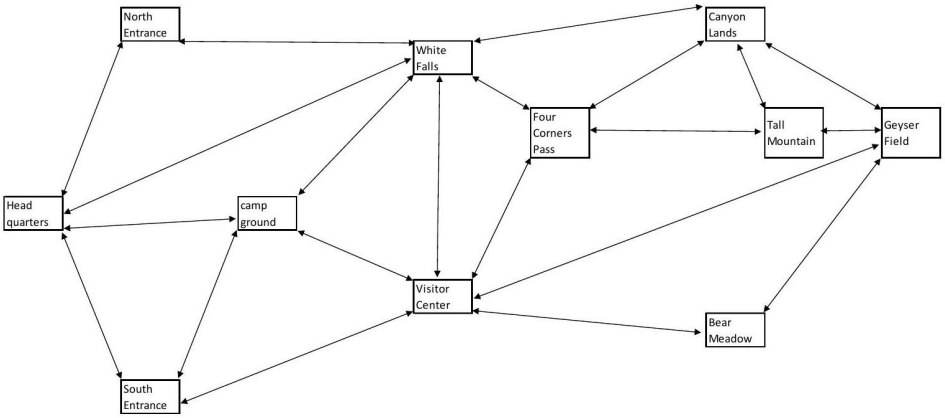
2. Traffic has gotten so bad that the park is considering banning personal vehicles on some roads and using shuttle busses instead. What should the bus routes be? How many busses are needed? Is a shuttle service even feasible?

3. During winter, some of the park roads will be closed. However, enough roads need to be plowed to allow access to all ranger stations. Which roads should be plowed?

4. A delivery truck needs to visit each of the parks location. What is the best route to take?

5. All roads need to be patrolled daily. What is the best route for the ranger to take?

Following is a simplified park map showing roads and attractions. As needed, you will also be provided with information regarding length, driving time, and shuttle capacity for each road segment.



References

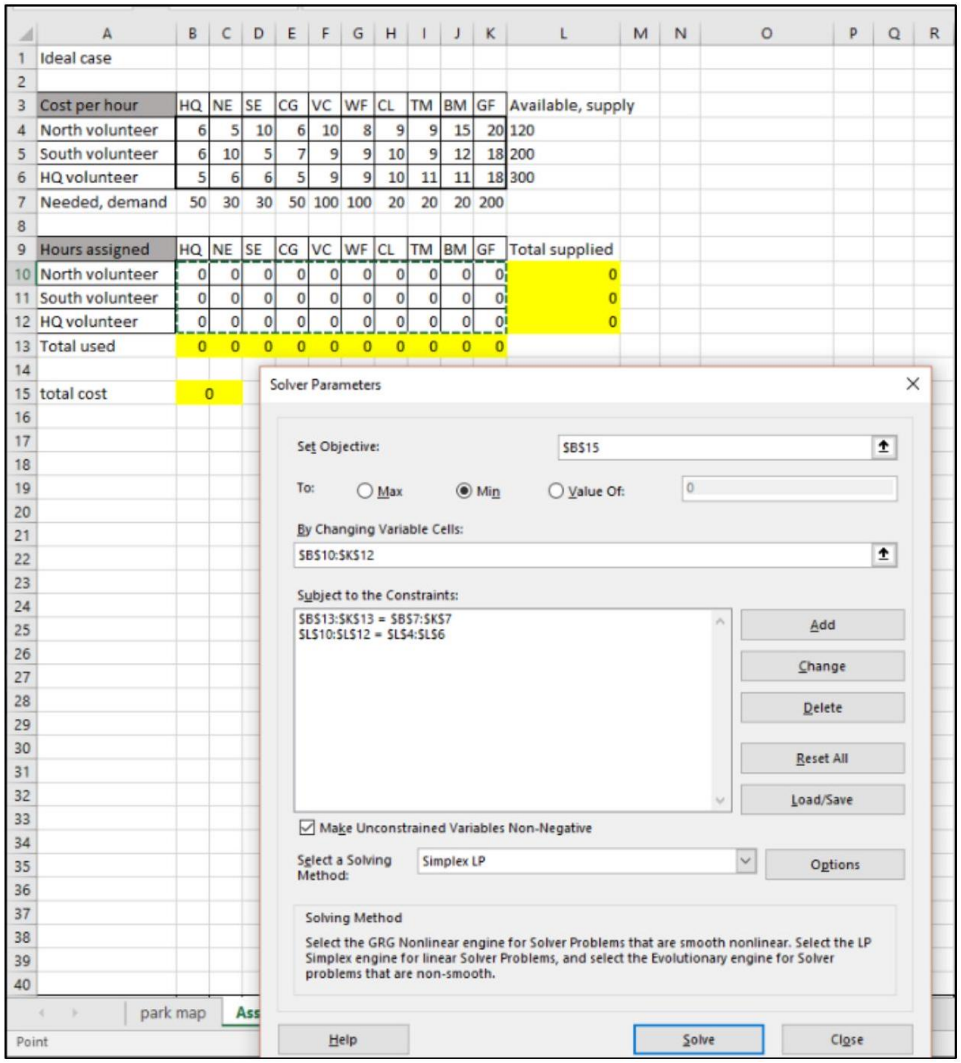
Budget cuts and overcrowding are very real issues for our National Parks. For more information on both, see:
https://www.npca.org/articles/1500-budget-proposal-threatens-nationalparks#sm.0000ks3wtu35mdezu
http://www.ournationalparks.us/park issues/busy parks seek approaches to manag e crowds cars/ https://www.nps.gov/zion/planyourvisit/traffic.htm
http://www.triplepundit.com/2016/03/national-parks-face-crowding-degradation/
http://www.hen.org/articles/arches-crowds-tourism-national-parks-utah https://www.nps.gov/yell/pla

Part 1: Work Force - Resource Allocation

Let's assume volunteers are available at the entrances to the park, HQ, NE, and SE . From there, you will have to transport them to the different locations and tasks. There is trail construction going on at VC and GF, so a lot of volunteers are needed there. GF is a very popular tourist destination, so lots of help is needed there as well. The cost/volunteer hour depends on a number of factors, such as distance to park entrance, nature of work, insurances required, etc. This table shows a summary of cost, supply, and demand.
We will denote each location by its initials, for example NE for North entrance, CL for Canyon lands etc. Four corners pass does not need any staffing, so we omit it.
This is an example of an allocation problem. A resource (volunteer hours, water rights, money, fish catch) is allocated to different areas (park location, cities, departments, canneries). In the ideal case, supply matches demand exactly. Let's start by assuming this is happening here:

Cost per hour	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF	Supply
North volunteer	6	5	10	6	10	8	9	9	15	20	120
South volunteer	6	10	5	7	9	9	10	9	12	18	200
HQ volunteer	5	6	6	5	9	9	10	11	11	18	300
Demand	50	30	30	50	100	100	20	20	20	200	

Note how the supply column and demand row add up to the same value, 620. Your objective is to minimize cost while meeting demand. Note that in this ideal case supply = demand. This problem can be set up as a linear programming problem and solved with the simplex method. You should check that the assumptions for a LP problem hold.
To solve this problem with Excel, we set up a second table containing the decision variables, i.e. how many hours from which source are being allocated to which area. Initially, we set them all to zero. The highlighted cells contain formulas. We can now use the Excel solver.



The solver returns the solution, which the park can then implement, and the total weekly cost of $6710.

Hours assigned	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF	Total supplied
North volunteer	0	0	0	0	0	100	20	0	0	0	120
South volunteer	0	0	30	0	0	0	0	20	0	150	200
HQ volunteer	50	30	0	50	100	0	0	0	20	50	300
Total used	50	30	30	50	100	100	20	20	20	200	
total cost	6710										

We were lucky and got integer value solutions. However, in this case the divisibility requirement of LP problems is satisfied, we could allow fractional hours worked.

Refinements

Let's assume you have too many volunteers, i.e. supply > demand. We incorporate them by assigning those hours we don't need to the "excess" column and proceeding otherwise just like before:

Cost per hour	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF	excess
North volunteer	6	5	10	6	10	8	9	9	15	20	0
South volunteer	6	10	5	7	9	9	10	9	12	18	0
HQ volunteer	5	6	6	5	9	9	10	11	11	18	0
Demand	50	30	30	50	100	100	20	20	20	200	130

supply
200
250
300

Hours assigned	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF	excess
North volunteer	0	0	0	0	0	0	0	0	0	0	0
South volunteer	0	0	0	0	0	0	0	0	0	0	0
HQ volunteer	0	0	0	0	0	0	0	0	0	0	0
Total used	0	0	0	0	0	0	0	0	0	0	0

Total supplied
0
0
0

```
total cost \
```

The table below shows the solution. Not surprisingly, with more volunteers available the overall cost went down.

Hours assigned	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF	not used
North volunteer	0	30	0	0	0	100	20	20	0	0	30
South volunteer	0	0	30	0	100	0	0	0	0	20	100
HQ volunteer	50	0	0	50	0	0	0	0	20	180	0

total cost	6680

If we have not enough volunteers, i.e. supply $<$ demand, we adjust the table by introducing a row "missing volunteers". Let's also assume that the visitor center absolutely needs all its requested volunteers. We achieve that by making the cost for the "missing volunteers" \$50000, so high that that will not be an option.

Cost per hour	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF
North volunteer	6	5	10	6	10	8	9	9	15	20
South volunteer	6	10	5	7	9	9	10	9	12	18
HQ volunteer	5	6	6	5	9	9	10	11	11	18
Missing	0	0	0	0	50000	0	0	0	0	0
Demand	50	30	30	60	100	100	30	20	20	200

Supply
120
200
300
20

Hours assigned	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF
North volunteer	0	0	0	0	0	90	30	0	0	0
South volunteer	0	0	30	0	100	10	0	20	0	40
HQ volunteer	50	30	0	60	0	0	0	0	20	140
Missing	0	0	0	0	0	0	0	0	0	20
Total used	50	30	30	60	100	100	30	20	20	200

Total supplied
120
200
300
20

total cost	6500

Next consider the case where GF, not happy with losing 20 volunteers, specifies both the minimum supply and ideal supply. We again make it prohibitively expensive to not give them the minimum.

Cost per hour	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF min	GF+
North volunteer	6	5	10	6	10	8	9	9	15	20	20
South volunteer	6	10	5	7	9	9	10	9	12	18	18
HQ volunteer	5	6	6	5	9	9	10	11	11	18	18
Missing	0	0	0	0	50000	0	0	0	0	50000	0
Demand	50	30	30	60	100	100	30	20	20	190	10

Supply
120
200
300
20

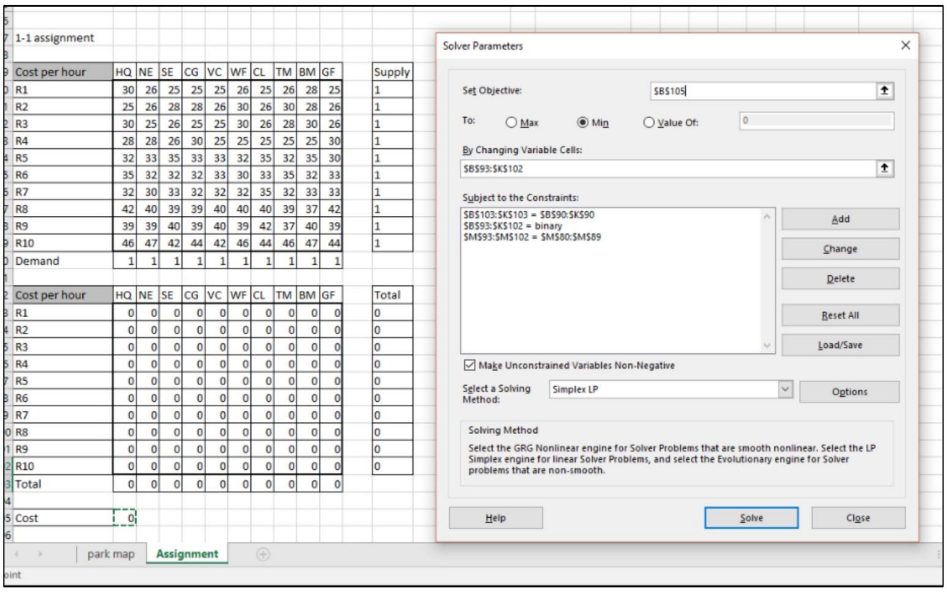
Hours assigned	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF min	GF+
North volunteer	0	30	0	0	0	60	30	0	0	0	0
South volunteer	0	0	30	0	100	40	0	20	0	10	0
HQ volunteer	50	0	0	60	0	0	0	0	10	180	0
Missing	0	0	0	0	0	0	0	0	10	0	10
Total used		50	30	30	60	100	100	30	20	20	190
10											

Total supplied
120
200
300
20

total cost	6570

Finally, we need to assign exactly one ranger to each location. This is an example for an assignment problem, rather than having a resource that can be divided up, we have to make 1-1 assignments. We do this by setting both supply and demand totals to one, and using the solver constraints to

set each decision variable to binary. If a certain ranger can't or won't work in a location, we just make the hourly cost too expensive for that particular assignment.



Cost per hour	HQ	NE	SE	CG	VC	WF	CL	TM	BM	GF
R1	0	0	1	0	0	0	0	0	0	0
R2	1	0	0	0	0	0	0	0	0	0
R3	0	0	0	1	0	0	0	0	0	0
R4	0	0	0	0	0	0	1	0	0	0
R5	0	0	0	0	0	0	0	0	0	1
R6	0	0	0	0	0	1	0	0	0	0
R7	0	1	0	0	0	0	0	0	0	0
R8	0	0	0	0	0	0	0	0	1	0
R9	0	0	0	0	0	0	0	1	0	0
R10	0	0	0	0	1	0	0	0	0	0
Total	1	1	1	1	1	1	1	1	1	1

Total
1
1
1
1
1
1
1
1
1
1

Cost	306

If we had more rangers than needed, we would again add a placeholder column "not needed" and assign one of the rangers to it. If we had too few rangers, one of the locations would not get staffed, i.e. we'd have an option "not staffed" as an additional row.

Stating the Solution

To minimize cost, rangers and volunteers should be assigned as follows (assuming exactly enough volunteers:

R1 → SE
R2 → HQ
R3 → CG
R4 → CL
R5 → GF
R6 → WF
R7 → NE
R8 → BM
R9 → TM
R10 → VC

Assign the 120 NE volunteer hours as follows: 100 hours to WF, 20 hours to CL
Assign the 200 SE volunteer hours as follows: 30 hours to SE, 150 hours to GF
Assign the 300 NE volunteer hours as follows: 50 hours to HQ, 30 hours to NE, 50 hours to CG, 100 hours to VC, 20 hours to BM, and 50 hours to GF

The total cost is $\$6,710$ / hour for the volunteers and \$306 / hour for the rangers.

Note that a similar set-up could be used to instead focus on worker preferences for assignments, their suitability for a task by ranking them, the driving distance to locations, or including urgency of assignments.

Part 2: Shuttle service - Shortest Path and Maximum Flow

We will examine two questions:

1. What is the shortest route in terms of time from HQ to GF and
2. Given the limited capacity of the narrow roads in the park, what is the maximum number of shuttle busses we can get from HQ to GF per hour? How much is it going to cost us? And how many visitors can we transport per hour?

We will refer to the table on the next page listing distance, drive time, and capacity for each road segment in the park. Assume that driving time is the same in either direction, and that capacity is as well (i.e. we can run the full capacity in each direction). Instead of minimizing time, one could choose to maximize profit, maximize popularity, or minimize miles driven or incorporate constraints such as parking available at different locations or one-way roads.

Shortest Path

From	To	Miles	Minutes	Capacity
HQ	NE	15	20	30
	WF	20	30	20
	CG	5	15	10
	SE	13	22	30
NE	HQ	15	20	30
	WF	12	30	15
CG	HQ	5	15	10
	SE	7	14	10
	VC	5	20	5
	WF	9	15	8
SE	HQ	13	22	30
	CG	7	14	10
	VC	22	25	30
vC	SE	22	25	30
	CG	5	20	5
	WF	15	30	9
	FCP	8	15	5
	GF	32	45	20
	BM	16	20	30
WF	NE	12	30	15
	HQ	20	30	20
	CG	9	15	8
	VC	15	30	9
	FCP	5	15	3
	CL	15	20	12
FCP	WF	5	15	3
	VC	8	15	5
	TM	10	15	6
	CL	11	15	8
CL	WF	15	20	12
	FCP	11	15	8
	TM	4	20	2
	GF	16	28	8
TM	FCP	10	15	6
	CL	4	20	2
	GF	3	15	2
BM	VC	16	20	30
	GF	20	30	15
GF	CL	16	28	8
	TM	3	15	2
	vC	32	45	20
	BM	20	30	15

To find the shortest path from HQ to GF, we can start by deleting all the roads that are not needed. For example, if we start at HQ, we will not go back from NE to HQ. Similarly, once we reach GF, we are done and do not need to leave again. This step is not strictly necessary to solve our small problem, but it is good form.

We will adopt terminology you may be familiar with from Discrete Structure. What we have here is a network problem. A network consists of nodes or vertices, the locations, and arcs, the roads. A (directed) path from A to B is a sequence of connecting arcs where the direction is from A to B. The out-degree, deg $^{+}$of a node is the number of arcs originating at that node (leaving). The in-degree, deg; of a node is the number of arcs terminating at that node (entering). The net flow is deg $^{+}$- deg. For our first question, we are looking for a shortest path from HQ to GF.

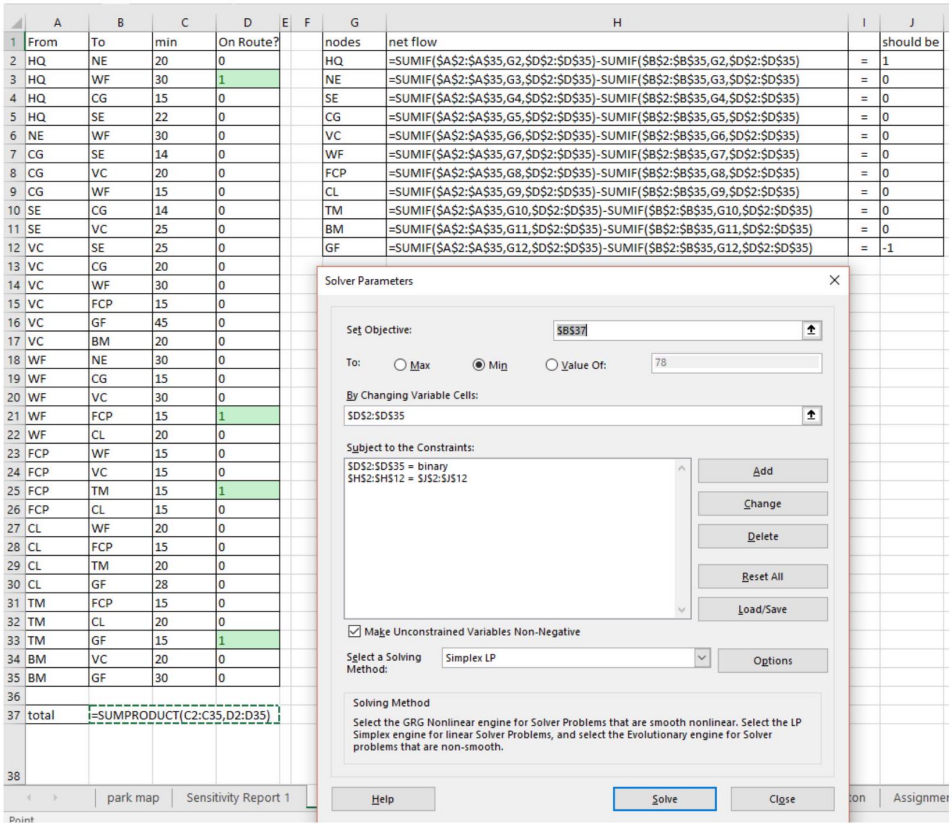
Because HQ is our start point, we want it to have a net flow of 1 . Our end point, GF, needs to have a net flow of -1, while all other nodes are either intermediate nodes or not used at all, so they all have zero net flow. You should convince yourself that this set-up guarantees a connected path from HQ to GF.

This problem can again be framed as a Linear Programming problem. We have a linear objective function to minimize, namely the sum of the times needed for all chosen paths. The decision variables, one for each path, have just two values. We set them = 1 if the path is chosen, and = 0 otherwise.

To compute net flow, we use the Sumlf function. The Sumlf function checks range_1 for any value that meets the criteria, and if it finds them adds up the corresponding entries in the sum_range.

Syntax: =Sumlf(range_1, criteria, sum_range)

For details on how to set up the work sheet, refer to the screenshot below. There are two important things: First of all, the decision variables need to be constraint to be binary. That way, we avoid partial roads, which make no sense. Second, it is important to hold the ranges in the "Sumlf" statement fixed (use F4 key or $$ signs)



From	To	min	On Route?
HQ	NE	20	0
HQ	WF	30	1
HQ	CG	15	0
HQ	SE	22	0
NE	WF	30	0
CG	SE	14	0
CG	VC	20	0
CG	WF	15	0
SE	CG	14	0
SE	VC	25	0
VC	SE	25	0
VC	CG	20	0
VC	WF	30	0
VC	FCP	15	0
VC	GF	45	0
VC	BM	20	0
WF	NE	30	0
WF	CG	15	0
WF	VC	30	0
WF	FCP	15	1
WF	CL	20	0
FCP	WF	15	0
FCP	VC	15	0
FCP	TM	15	1
FCP	CL	15	0
CL	WF	20	0
CL	FCP	15	0
CL	TM	20	0
CL	GF	28	0
TM	FCP	15	0
TM	CL	20	0
TM	GF	15	1
BM	VC	20	0
BM	GF	30	0
total	=SUMPRODUCT(C2:C35,D2:D35)		

nodes	net flow		should be
HQ	=SUMIF($A$2:$A$35,G2,$D$2:$D$35)-SUMIF($B$2:$B$35,G2,$D$2:$D$35)	=	1
NE	=SUMIF($A$2:$A$35,G3,$D$2:$D$35)-SUMIF($B$2:$B$35,G3,$D$2:$D$35)	=	0
SE	=SUMIF($A$2:$A$35,G4,$D$2:$D$35)-SUMIF($B$2:$B$35,G4,$D$2:$D$35)	=	0
CG	=SUMIF($A$2:$A$35,G5,$D$2:$D$35)-SUMIF($B$2:$B$35,G5,$D$2:$D$35)	=	0
VC	=SUMIF($A$2:$A$35,G6,$D$2:$D$35)-SUMIF($B$2:$B$35,G6,$D$2:$D$35)	=	0
WF	=SUMIF($A$2:$A$35,G7,$D$2:$D$35)-SUMIF($B$2:$B$35,G7,$D$2:$D$35)	=	0
FCP	=SUMIF($A$2:$A$35,G8,$D$2:$D$35)-SUMIF($B$2:$B$35,G8,$D$2:$D$35)	=	0
CL	=SUMIF($A$2:$A$35,G9,$D$2:$D$35)-SUMIF($B$2:$B$35,G9,$D$2:$D$35)	=	0
TM	=SUMIF($A$2:$A$35,G10,$D$2:$D$35)-SUMIF($B$2:$B$35,G10,$D$2:$D$35)	=	0
BM	=SUMIF($A$2:$A$35,G11,$D$2:$D$35)-SUMIF($B$2:$B$35,G11,$D$2:$D$35)	=	0
GF	=SUMIF($A$2:$A$35,G12,$D$2:$D$35)-SUMIF($B$2:$B$35,G12,$D$2:$D$35)	=	-1

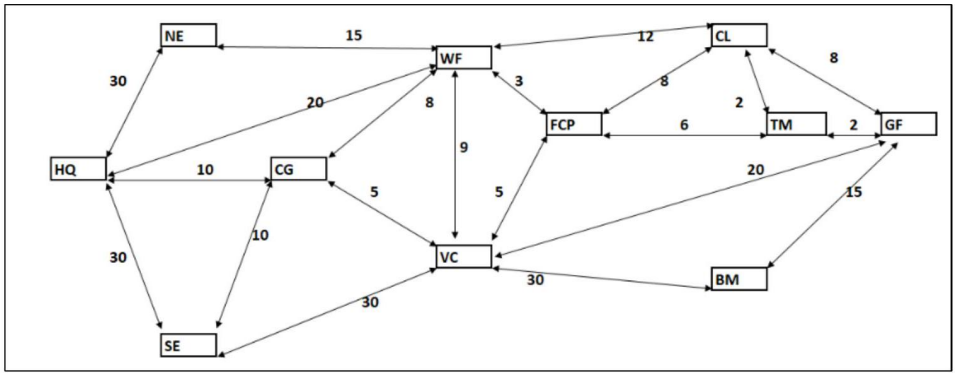
Stating the Solution

The solution turns out HQ $\rightarrow$ WF $\rightarrow$ FCP $\rightarrow$ TM $\rightarrow$ GF for a total of 75 minutes driving time. This is the shortest path (in terms of time) available. As far as the capacity is concerned, we need to use the minimum capacity for any of the four legs, which is 2 for TM to GF.

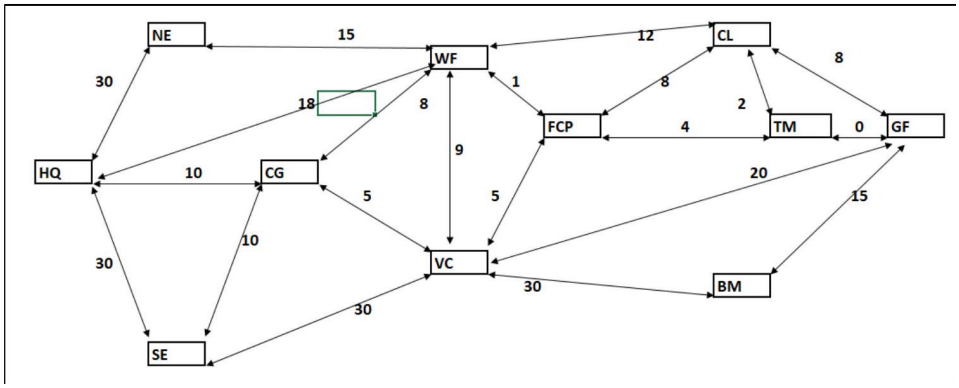
Maximum Flow

Two shuttles are not much, we will probably need more. How many shuttle busses total could we get from HQ to GF? And how can we do that with minimum total time? This question really asks us to solve
a problem with two objective functions: Maximize flow while minimizing total time. To answer that question, we look at our park map, this time

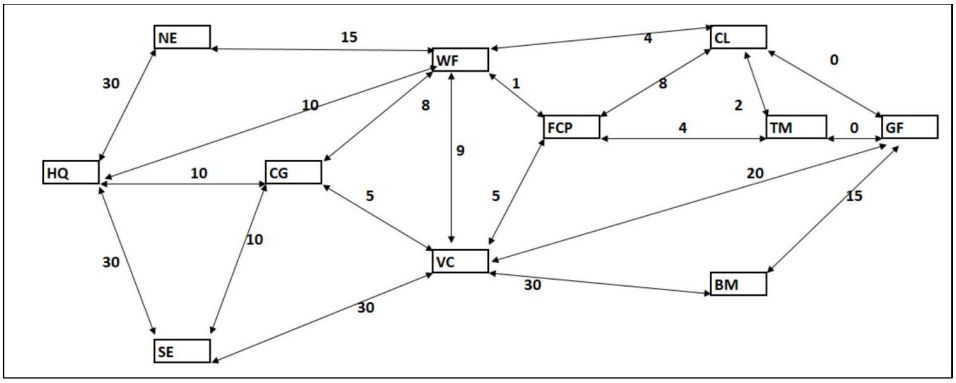
annotated with the capacities.



We already run 2 shuttles on HQ $\rightarrow$ WF $\rightarrow$ FCP $\rightarrow$ TM $\rightarrow$ GF, so we change the numbers to reflect capacities still available. In effect, the road from TM to GF is no longer available. We eliminate that option from our list. (line 33).



Running the shortest path problem again (with one road less) gives us the path $HQ \rightarrow WF \rightarrow CL \rightarrow F \rightarrow GF$, with 78 minutes. This route will accommodate 8 additional shuttles. We again adjust our map and table to reflect that yet another route is maxed out.

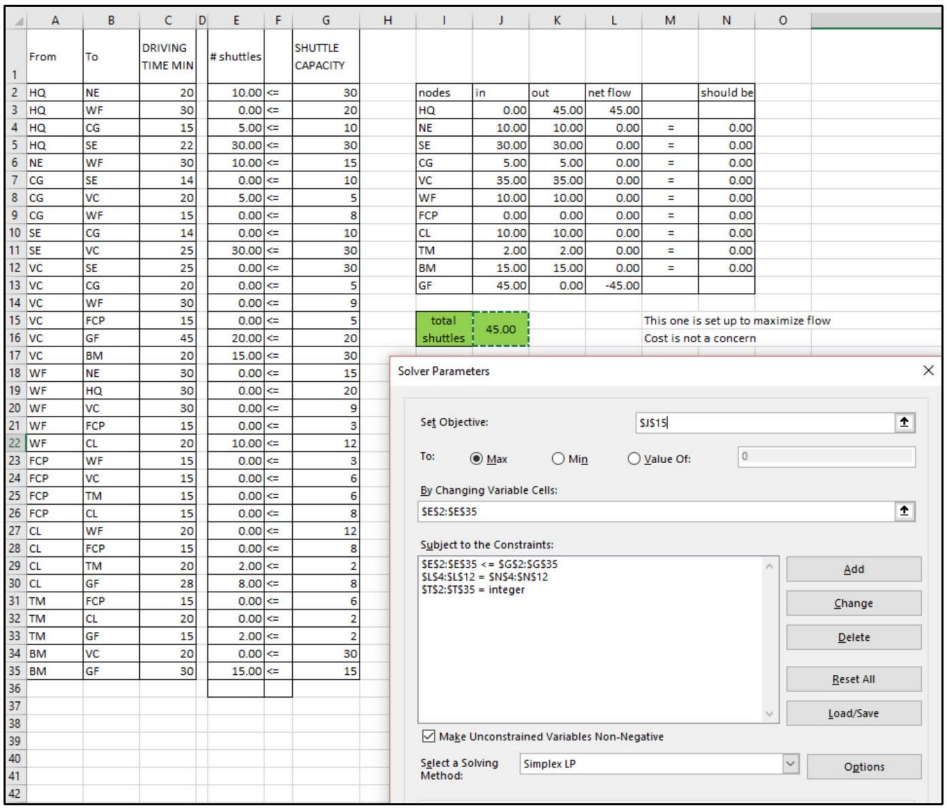


We continue alternating the shortest path problem with adjusting route list and map, and arrive at the solution:

route	time	capacity
HQ → WF → FCP → TM → GF	75	2
HQ → WF → CL → GF	78	8
HQ → CG → VC → GF	80	5
HQ → SE → VC → GF	92	15
HQ → SE → VC → BM → GF	97	15
	Average time ≈ 89 min	Total shuttles: 45

This is an example of a greedy algorithm. In each iteration, we pick what is best without considering the impact on future iterations. A greedy algorithm may, depending on the problem, have the effect that each iteration gives the optimal result given the previous iterations, but not necessarily the best result overall.

Following is a different, two-step approach. First, we solve the maximum flow problem without cost concerns. We use the max capacity as one of our constraints and allow the decision variables to be integers. The net flow is restricted only for the transitional nodes.



From	To	DRIVING TIME MIN	# shuttles		SHUTTLE CAPACITY
HQ	NE	20	10.00	<=	30
HQ	WF	30	0.00	<=	20
HQ	CG	15	5.00	<=	10
HQ	SE	22	30.00	<=	30
NE	WF	30	10.00	<=	15
CG	SE	14	0.00	<=	10
CG	VC	20	5.00	<=	5
CG	WF	15	0.00	<=	8
SE	CG	14	0.00	<=	10
SE	VC	25	30.00	<=	30
VC	SE	25	0.00	<=	30
VC	CG	20	0.00	<=	5
VC	WF	30	0.00	<=	9
VC	FCP	15	0.00	<=	5
VC	GF	45	20.00	<=	20
VC	BM	20	15.00	<=	30
WF	NE	30	0.00	<=	15
WF	HQ	30	0.00	<=	20
WF	VC	30	0.00	<=	9
WF	FCP	15	0.00	<=	3
WF	CL	20	10.00	<=	12
FCP	WF	15	0.00	<=	3
FCP	VC	15	0.00	<=	6
FCP	TM	15	0.00	<=	6
FCP	CL	15	0.00	<=	8
CL	WF	20	0.00	<=	12
CL	FCP	15	0.00	<=	8
CL	TM	20	2.00	<=	2
CL	GF	28	8.00	<=	8
TM	FCP	15	0.00	<=	6
TM	CL	20	0.00	<=	2
TM	GF	15	2.00	<=	2
BM	VC	20	0.00	<=	30
BM	GF	30	15.00	<=	15

nodes	in	out	net flow		should be
HQ	0.00	45.00	45.00		
NE	10.00	10.00	0.00	=	0.00
SE	30.00	30.00	0.00	=	0.00
CG	5.00	5.00	0.00	=	0.00
VC	35.00	35.00	0.00	=	0.00
WF	10.00	10.00	0.00	=	0.00
FCP	0.00	0.00	0.00	=	0.00
CL	10.00	10.00	0.00	=	0.00
TM	2.00	2.00	0.00	=	0.00
BM	15.00	15.00	0.00	=	0.00
GF	45.00	0.00	-45.00		

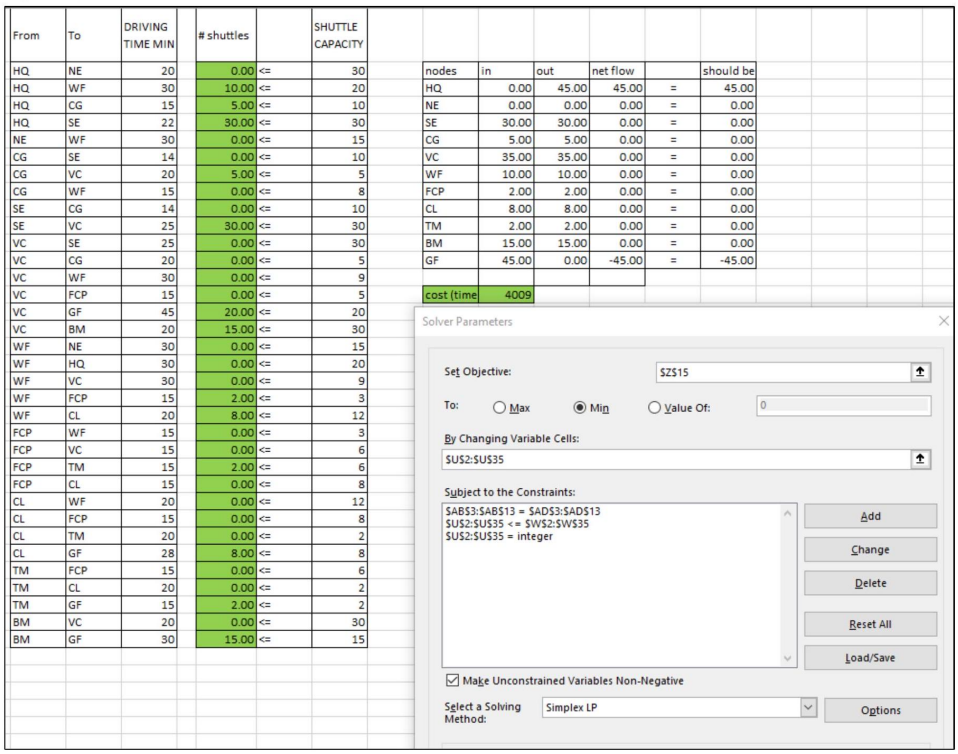
total shuttles 45.00

This one is set up to maximize flow
Cost is not a concern

We find that the maximum possible flow is 45 , just as before. Next, we can address cost (time).

Maximum Flow - Minimum Cost

To solve the maximum flow problem with cost constraints we use the result from the last section and modify the minimum cost problem. We set the netflow for our start point, HQ , to the maximum capacity and the netflow of our end point, GF, to the negative maximum capacity. Of course, we also change the objective function.



From	To	DRIVING TIME MIN	# shuttles		SHUTTLE CAPACITY
HQ	NE	20	0.00	<=	30
HQ	WF	30	10.00	<=	20
HQ	CG	15	5.00	<=	10
HQ	SE	22	30.00	<=	30
NE	WF	30	0.00	<=	15
CG	SE	14	0.00	<=	10
CG	VC	20	5.00	<=	5
CG	WF	15	0.00	<=	8
SE	CG	14	0.00	<=	10
SE	VC	25	30.00	<=	30
VC	SE	25	0.00	<=	30
VC	CG	20	0.00	<=	5
VC	WF	30	0.00	<=	9
VC	FCP	15	0.00	<=	5
VC	GF	45	20.00	<=	20
VC	BM	20	15.00	<=	30
WF	NE	30	0.00	<=	15
WF	HQ	30	0.00	<=	20
WF	VC	30	0.00	<=	9
WF	FCP	15	2.00	<=	3
WF	CL	20	8.00	<=	12
FCP	WF	15	0.00	<=	3
FCP	VC	15	0.00	<=	6
FCP	TM	15	2.00	<=	6
FCP	CL	15	0.00	<=	8
CL	WF	20	0.00	<=	12
CL	FCP	15	0.00	<=	8
CL	TM	20	0.00	<=	2
CL	GF	28	8.00	<=	8
TM	FCP	15	0.00	<=	6
TM	CL	20	0.00	<=	2
TM	GF	15	2.00	<=	2
BM	VC	20	0.00	<=	30
BM	GF	30	15.00	<=	15

nodes	in	out	net flow		should be
HQ	0.00	45.00	45.00	=	45.00
NE	0.00	0.00	0.00	=	0.00
SE	30.00	30.00	0.00	=	0.00
CG	5.00	5.00	0.00	=	0.00
VC	35.00	35.00	0.00	=	0.00
WF	10.00	10.00	0.00	=	0.00
FCP	2.00	2.00	0.00	=	0.00
CL	8.00	8.00	0.00	=	0.00
TM	2.00	2.00	0.00	=	0.00
BM	15.00	15.00	0.00	=	0.00
GF	45.00	0.00	-45.00	=	-45.00
cost (time	4009				

The average time needed is the same as before, ≈ 90 minutes per shuttle. Note that you could use this last set-up to find the lowest average time for any number of shuttles, as long as the total stays at or below capacity.

While the total number of shuttles and the time needed is the same for both approaches, the routes taken are different. This is common for maximum flow problems. In fact, if you choose to solve a maximum flow problem by hand, it does not matter which route you start with.

Stating the Solution

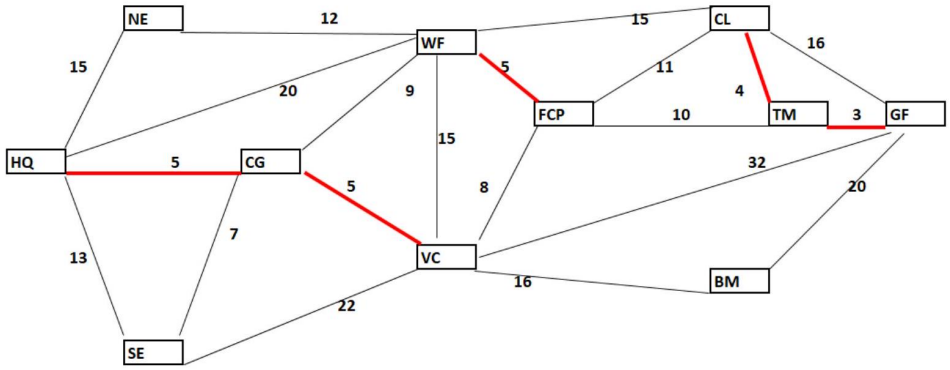
If we use 50 passenger busses, we can transport 2250 passengers per hour using the maximum possible 45 shuttles per hour leaving HQ. Given that the roundtrip takes and average 90 minutes, and allowing 90 minutes at GF for sightseeing, this probably means we should start leaving HQ at 8 am and have the last shuttle leave no later than 4 pm . This means we need 135 busses and can transport 18,000 passengers per day. This may not be enough (given that, for example, Yellowstone sees 900,000+ visitors in July). We may have to consider bigger busses, wider roads, or restricting

visitor numbers.

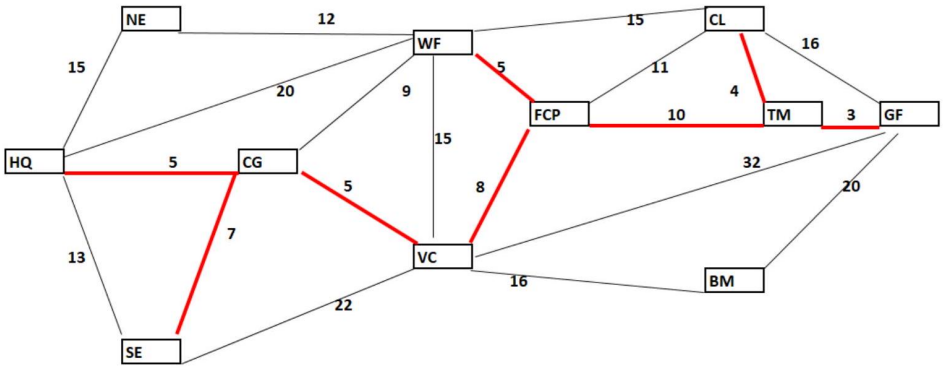
Part 3: Plowing - Minimum Spanning Tree

This problem is an example of a minimum spanning tree problem. A minimum spanning tree is a subset of a connected, weighted, undirected graph that connects all the nodes together in such a way that the total arc weight (cost) is minimized. There can be no cycles or loops, otherwise the tree would not be minimal.

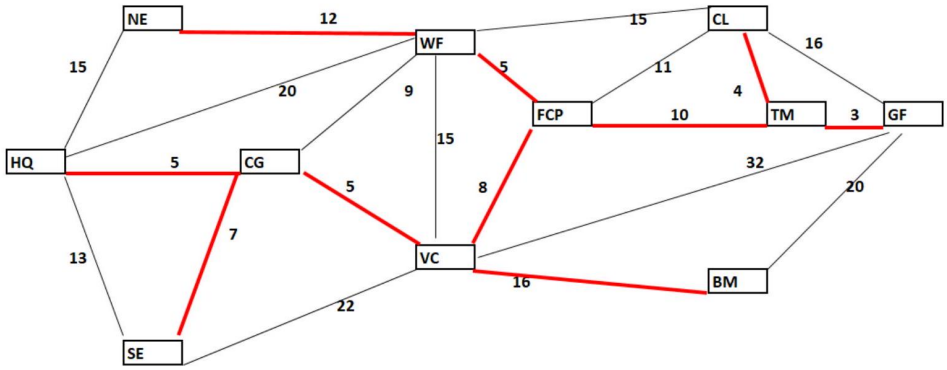
Our plowing contractor charges by the mile, so we are trying to minimize distance, not driving time. Each road needs two passes, one in each direction. Here again a greedy algorithm works. Start with a graph showing all the locations (nodes), roads (arcs), and distances (weights). Build the tree by including the shortest roads first, but only if they do not create any loops. Add the next shortest roads, again watching out for loops. Continue until you have a connected tree.
We will add roads TM $\rightarrow$ GF(3), CL $\rightarrow$ TM(4), WF $\rightarrow$ CL(5), HQ $\rightarrow$ CG(5), CG $\rightarrow$ VC (5) successively.



Next are SE $\rightarrow$ CG (7) and VC $\rightarrow$ FCP (8), but not CG $\rightarrow$ WF (9) which would create a loop. We can add TM $\rightarrow$ FCP (10), but not FCP $\rightarrow$ CL (11).



Finally, we add $WF \rightarrow NE$ (12) and $VC \rightarrow BM$ (16), for a total of 75 miles of road or 150 miles of plowing.



This also tells us which roads can be closed in winter. Because we found a spanning tree, the contractor can start and end at any location or entrance.

Stating the Solution

To minimize cost, the following roads should be plowed:
TM $\rightarrow$ GF, CL $\rightarrow$ TM, WF $\rightarrow$ CL, HQ $\rightarrow$ CG, CG $\rightarrow$ VC, WF $\rightarrow$ NE, SE $\rightarrow$ CG, VC $\rightarrow$ FCP, TM $\rightarrow$ FCP, and VC $\rightarrow$ BM, for a total of 150 miles of plowing. Contractors located at any of the three park entrances can be used with no difference in mileage, so the cheapest option should be used

Part 4: Deliveries - Traveling Salesman Problem

Assume you need to deliver supplies to each of the 11 locations in the park, starting and ending at HQ . You want to visit each location exactly once. In graph theory, a path that visits each vertex in a graph exactly once and ends

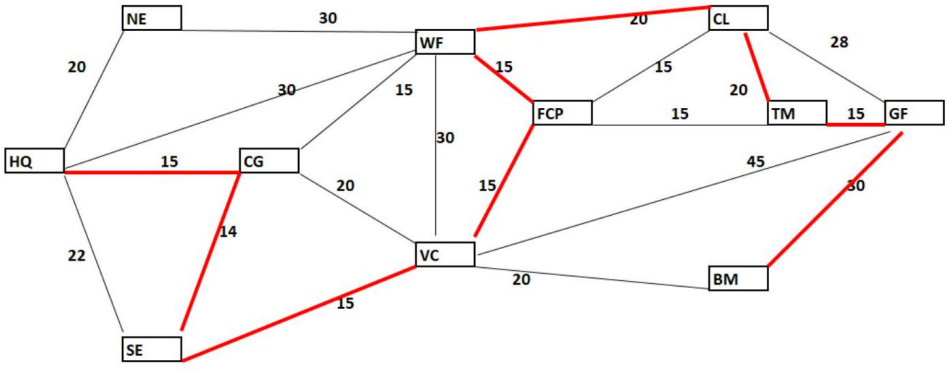
and starts at the same vertex is called a Hamiltonian circuit. The problem of finding the minimum cost such path is called the Traveling Salesman Problem (TSP), probably the most famous problem in OR. This is a notoriously difficult problem to solve efficiently.

There are 11 locations to visit in a circuit. One could attempt to list all possible paths of length 11, pick out the potential solutions (those through all 11 locations), compute the length and pick the best. The problem with this brute force approach is that there are so many possibilities. In the worst-case scenario, i.e. a complete graph with 11 vertices (complete means that each vertex is connected to every other vertex), there are $\frac{(n-1)!}{2} = \frac{10!}{2} = 1,814,400$ possible paths to check. While we do have only 21 edges instead of the 55 of a complete graph, it is still not feasible to find and check all potential paths.

We will use an approach called a heuristic, which is short for heuristic technique. A heuristic is an approach that uses a practical, often intuitive approach to problem solving. It is not guaranteed to yield the optimal result, but instead one that is "good enough" (hopefully),

Nearest neighbor approach

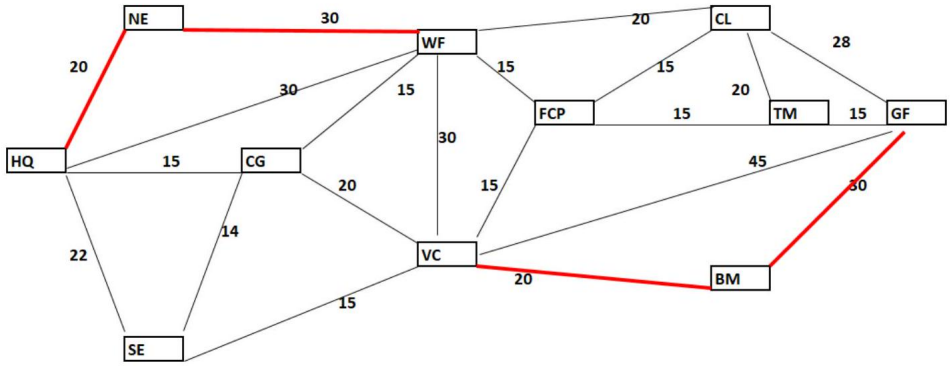
The nearest neighbor approach is fast, simple, and often yields a good short path. However, there are instances where it doesn't work at all or even yields the worst path. The idea is to start at an arbitrary vertex, chose the edge to the nearest unconnected vertex, and continue until the circuit is complete. This approach works well if the graph is complete or nearly so. In our case, starting at HQ , we end up "stuck", there is no path from BM back to NE:



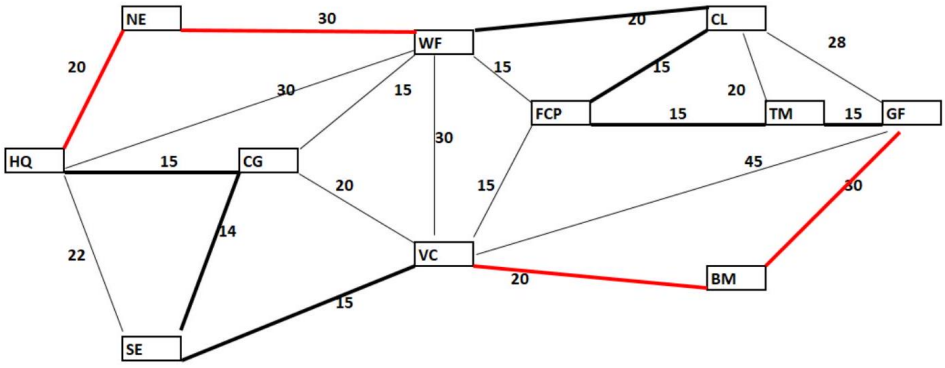
You may have noticed that we had choices at FCP, however, each of them will lead to the same problem. There just aren't enough edges.

One way around this is to use a modified nearest neighbor approach. We recognize that there are two nodes, NE and BM, where we do not have a

choice how to reach them. We can therefor treat those segments (shown below) as a unit and try the nearest neighbor approach again.

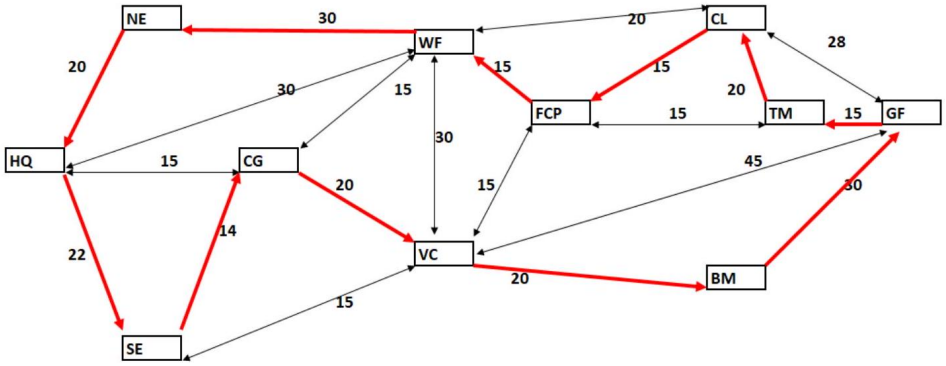


Starting at HQ , this time we get a solution. The total length of this circuit is 209.



Sub-tour Reversal

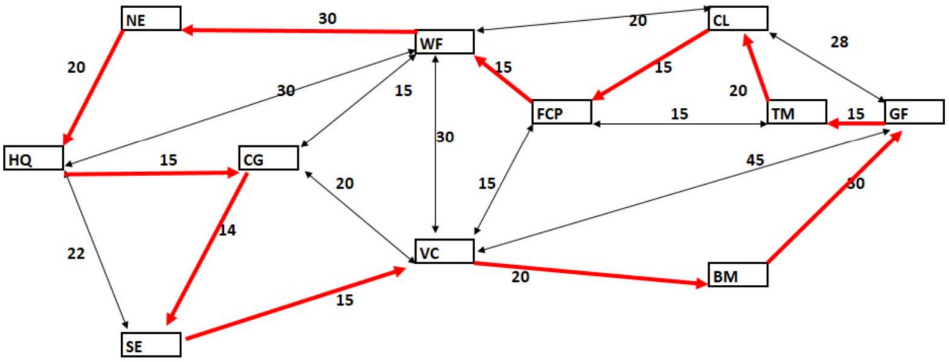
Next, we examine a technique called sub-tour reversal. We start with any possible directed circuit:



This circuit is not - and doesn't have to be - perfect, it just serves as a starting point. The idea is to look at sub-tours, starting with one-segment ones, then two-segments, etc. We will look at each of those sub-tours and check if the total cost can be improved if we reverse direction. If we reverse direction, it will also affect the two path segments directly adjacent. We again start at HQ.

HQ → SE	If we reverse HQ → SE, the edges NE → HQ and SE → CG no longer worl
SE → CG	Reversing SE → CG to CG → SE means the adjacent edges, HQ → SE and SE → VC
CG → VC	N
VC → BM	N
BM → GF	N
GF → TM	N
TM → CL	This reversal is po
CL → FCP	This reversal is
FCP → WF	N
WF → NE	N
NE → HQ	N

Of all three possible 1-segment subtour reversals, reversing $SE \rightarrow CG$ to $CG \rightarrow$ SE gives the best result, so we will implement that. Next, we check all two-segment sub-tours, then all three-segment sub-tours and so on (it turns out none of those are reversible). Our final circuit looks like this:



TSP using Excel

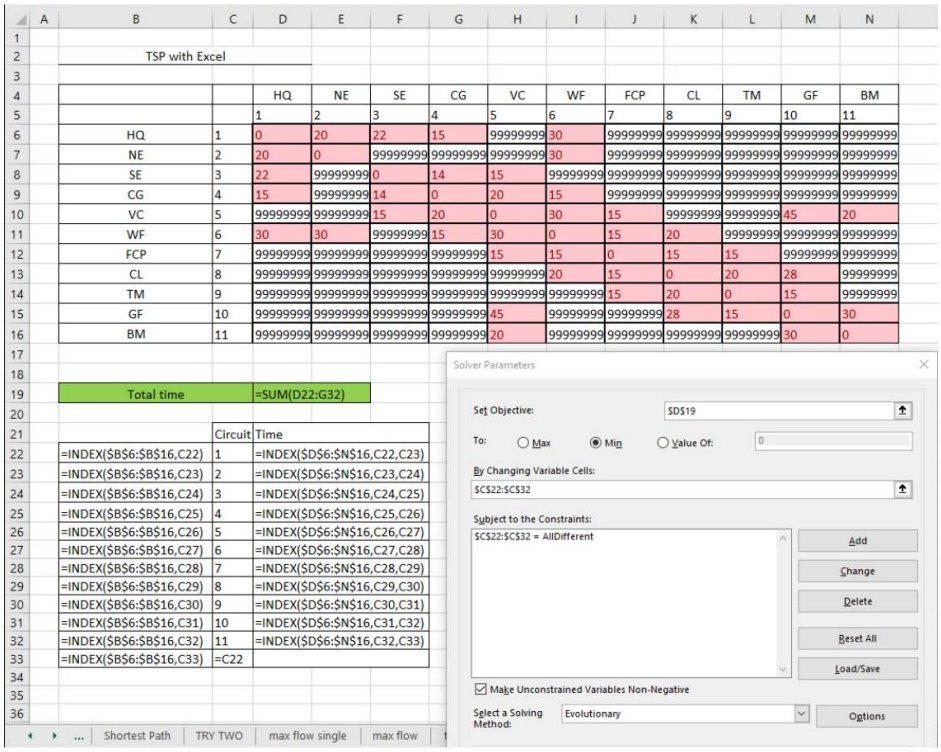
To use Excel to solve the TSP problem, we first have to set up the distance matrix. It shows the distance in minutes between each pair of nodes. As we have done earlier, we set this value to a large number when no such path exists to make sure that option is never chosen. Note that the matrix is symmetrical, this is because all our roads can be driven in either direction. If we had one-way roads the matrix would no longer be symmetrical.

This is no longer a linear programming problem. For example, we assign the different nodes arbitrary numbers, so the contribution of the decision variables to the value of the objective function is no longer proportional to their value. Excel offers two other solver options: A generalized reduced gradient algorithm and an evolutionary algorithm. The details of these algorithms are a bit beyond this class, though I am happy to talk to you outside the class room if you are interested. Basically, the GRG method finds the direction of steepest increase or decrease to move towards the maximum/minimum. It works well if the objective functions and feasible region are smooth-ish (think parabolas, cones, etc.). The evolutionary algorithm does not have any smoothness requirements. It uses a random start point and then evolves the next generation of solution points from that start point (a little like the path-reversal, but with a strong random element). It does not use gradient information, so you cannot be sure you reached the best solution. The evolutionary solver will keep searching for better solutions until it no longer makes sufficient progress or runs out of time. We will use the Evolutionary Solver option.

We start with an arbitrary sequence 1-2-3-4-5-6-7-8-9-10-11-1. The INDEX function is used to look the associated time in the distance matrix.

Syntax: =INDEX(range where to look, what row to use, optional: what

column to use)
The objective function is simply the sum of the times. As our only constraint we use the "all different" option. The constraint "$C$22:$C$32 = AllDifferent" means that the 11 decision variables stored in $\$C\$22 : \$C\$32$ must be integers in the range 1-11, with each one different from all the others, i.e. we require permutations of $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$. Finally, you want to choose the option to set a time limit for your solver, 60 seconds should be sufficient.



The solver returns the solution below. The time requirement is the same as we found in our previous attempts.

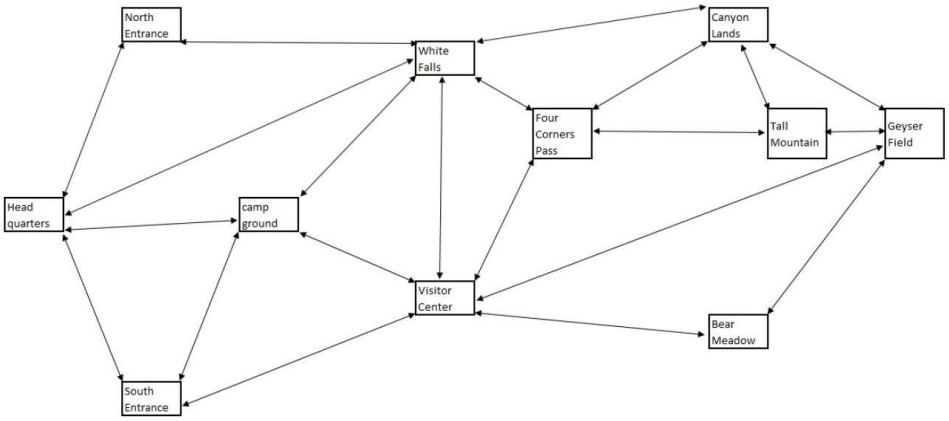
	Circuit	Time
WF	6	20
CL	8	15
FCP	7	15
TM	9	15
GF	10	30
BM	11	20
VC	5	15
SE	3	14
CG	4	15
HQ	1	20
NE	2	30
WF	6	

Stating the Solution

Starting at HQ, the route traveled will take 209 minutes or 3 hours 29 minutes. An optimal route to take is HQ $\rightarrow$ NE $\rightarrow$ WF $\rightarrow$ CL $\rightarrow$ FCP $\rightarrow$ TM $\rightarrow$ GF $\rightarrow$ BM $\rightarrow$ VC $\rightarrow$ SE $\rightarrow$ CG $\rightarrow$ HQ. An alternate of same length is HQ $\rightarrow$ NE $\rightarrow$ WF $\rightarrow$ FCP $\rightarrow$ CL $\rightarrow$ TM $\rightarrow$ GF $\rightarrow$ BM $\rightarrow$ VC $\rightarrow$ SE $\rightarrow$ CG $\rightarrow$ HQ, and of course both routes can be reversed.

Part 5: Road Patrols - Traversing

To ensure the safety of visitors, a ranger has to patrol the entire road system in the park daily, starting and ending at HQ while minimizing travel time. If possible, they should drive each road only once in either direction. In graph theory, a path that visits each arc in a graph exactly once and ends and starts at the same vertex is called a Eulerian circuit. You may recall from Discrete Mathematics that a Euler circuit exists if and only if the degree of each vertex is even, i.e. an even number of edges connect to each vertex. Unfortunately, that is not the case here, SE and TM both have degree 3.



We already know that it is not possible to traverse each road exactly once, so the question now becomes how many times each road needs to be traveled, and in which direction. We re-arrange our table to list both road directions next to each other. This makes it easier to write the constraints: "each segment needs to be traveled at least once in either direction".

From	To	Time/min	Times traveled
HQ	NE	20	1
NE	HQ	20	0
HQ	WF	30	0
WF	HQ	30	1
HQ	CG	15	1
CG	HQ	15	0
HQ	SE	22	0
SE	HQ	22	1
NE	WF	30	1
WF	NE	30	0
CG	SE	14	1
SE	CG	14	0
CG	VC	20	0
VC	CG	20	1
CG	WF	15	1
WF	CG	15	0
SE	VC	25	1
VC	SE	25	1
VC	WF	30	1
WF	VC	30	0
VC	FCP	15	0
FCP	VC	15	2
VC	GF	45	1
GF	VC	45	0
VC	BM	20	0
BM	VC	20	1
WF	FCP	15	1
FCP	WF	15	0
WF	CL	20	1
CL	WF	20	0
FCP	TM	15	1
TM	FCP	15	1
FCP	CL	15	0
CL	FCP	15	1
CL	TM	20	1
TM	CL	20	0
CL	GF	28	0
GF	CL	28	1
TM	GF	15	1
GF	TM	15	0
BM	GF	30	0
GF	BM	30	1

Segment	Total/segment		
HQ - NE	=D2+D3	>=	1
HQ - NWF	=D4+D5	>=	1
HQ - CG	=D6+D7	>=	1
HQ - SE	=D8+D9	>=	1
NE - WF	=D10+D11	>=	1
CG - SE	=D12+D13	>=	1
CG - VC	=D14+D15	>=	1
CG - WF	=D16+D17	>=	1
SE - VC	=D18+D19	>=	1
VC - WF	=D20+D21	>=	1
VC - FCP	=D22+D23	>=	1
VC - GF	=D24+D25	>=	1
VC - BM	=D26+D27	>=	1
WF - FCP	=D28+D29	>=	1
WF - CL	=D30+D31	>=	1
FCP - TM	=D32+D33	>=	1
FCP - CL	=D34+D35	>=	1
CL - TM	=D36+D37	>=	1
CL - GF	=D38+D39	>=	1
TM - GF	=D40+D41	>=	1
BM - GF	=D42+D43	>=	1

total time
=SUMPRODUCT(C2:C45,D2:D43)

nodes	in	out	net flow		should be
HQ	=SUMIF($B$2:$B$43,K3,$D$2:$D$43)	=SUMIF($A$2:$A$43,K3,$D$2:$D$43)	=M3-L3	=	0
NE	=SUMIF($B$2:$B$43,K4,$D$2:$D$43)	=SUMIF($A$2:$A$43,K4,$D$2:$D$43)	=M4-L4	=	0
SE	=SUMIF($B$2:$B$43,K5,$D$2:$D$43)	=SUMIF($A$2:$A$43,K5,$D$2:$D$43)	=M5-L5	=	0
CG	=SUMIF($B$2:$B$43,K6,$D$2:$D$43)	=SUMIF($A$2:$A$43,K6,$D$2:$D$43)	=M6-L6	=	0
VC	=SUMIF($B$2:$B$43,K7,$D$2:$D$43)	=SUMIF($A$2:$A$43,K7,$D$2:$D$43)	=M7-L7	=	0
WF	=SUMIF($B$2:$B$43,K8,$D$2:$D$43)	=SUMIF($A$2:$A$43,K8,$D$2:$D$43)	=M8-L8	=	0
FCP	=SUMIF($B$2:$B$43,K9,$D$2:$D$43)	=SUMIF($A$2:$A$43,K9,$D$2:$D$43)	=M9-L9	=	0
CL	=SUMIF($B$2:$B$43,K10,$D$2:$D$43)	=SUMIF($A$2:$A$43,K10,$D$2:$D$43)	=M10-L10	=	0
TM	=SUMIF($B$2:$B$43,K11,$D$2:$D$43)	=SUMIF($A$2:$A$43,K11,$D$2:$D$43)	=M11-L11	=	0
BM	=SUMIF($B$2:$B$43,K12,$D$2:$D$43)	=SUMIF($A$2:$A$43,K12,$D$2:$D$43)	=M12-L12	=	0
GF	=SUMIF($B$2:$B$43,K13,$D$2:$D$43)	=SUMIF($A$2:$A$43,K13,$D$2:$D$43)	=M13-L13	=	0

Solver Parameters

Set Objective: $F$25

To: Max Min Value Of: 0

By Changing Variable Cells: $D$2:$D$43

Subject to the Constraints:

$D$2:$D$43 = integer
$G$2:$G$22 >= $I$2:$I$22
$N$3:$N$13 = $P$3:$P$13

Add
Change
Delete
Reset All
Load/Save

Make Unconstrained Variables Non-Negative

Select a Solving Method: Simplex LP

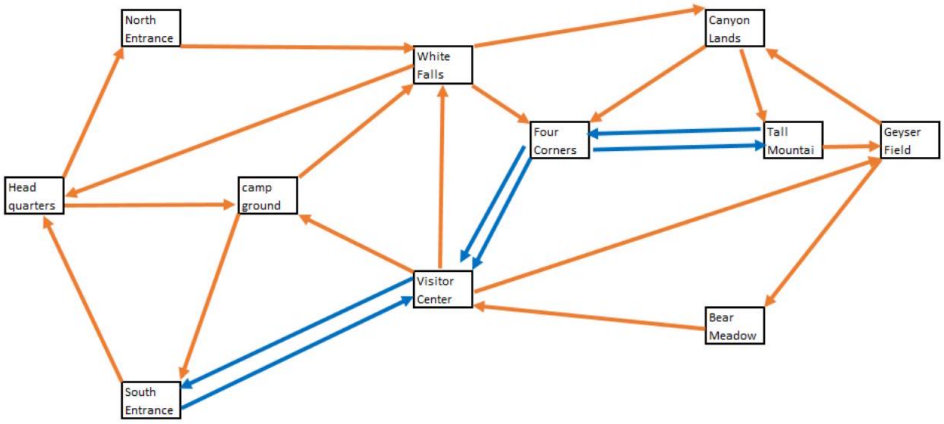
Options

Solving Method

Select the GRG Nonlinear engine for Solver Problems that are smooth nonlinear. Select the LP Simplex engine for linear Solver Problems, and select the Evolutionary engine for Solver problems that are non-smooth.

Stating the Solution

It will take a single ranger 514 minutes or 8 hours 34 minutes to patrol all the roads in the park. One possible optimal route is:
HQ → NE → WF → HQ → CG → WF → FCP → VC → WF → CL → FCP → TM → GF → CL → TM → FCP → VC → GF → BM → VC → CG → SE → VC → SE → HQ. This is just one of many options. The only rule is that either all roads have to be travelled in the direction and frequencies shown, or all directions have to be reversed.



The patrol as specified is not possible without overtime, especially not if anything needs attention. Possible alternatives are:

- Splitting the area between two or more patrols or two or more days
- Eliminating roads from the patrol
- Faster car?

Comment: Note that this is a Euler path if you eliminate the second traversals. Basically, we have a Euler path from SE to TM + the fastest way / shortest path linking the beginning and end of that Euler path. If you think of each arrow as an undirected arc, then all nodes have even degree and a Euler circuit exists. Basically, we figured out where edges need to be added to have the cheapest possible Euler path/circuit.